

Python

- [Pre-Req](#)
- [Windows](#)
 - [Install Python](#)
 - [Install requests module](#)
 - [Install requests-kerberos module](#)
 - [Set up SSL verification](#)
- [Linux](#)
 - [Installation](#)
 - [Authorization](#)
- [Sample Calls](#)
 - [How to do Basic Auth in Python](#)
 - [Kerberos auth for faceless account:](#)
 - [GET](#)
 - [Get an HSD Article](#)
 - [Run a query by ID](#)
 - [Run a query using EQL](#)
 - [POST](#)
 - [Insert a new HSD article](#)
 - [Insert a new comment for an existing Article](#)
 - [Update an existing HSD article](#)
 - [Upload an attachment](#)
 - [Download an attachment](#)
 - [Adding embedded images to description](#)
- [Support](#)

Pre-Req

Note: Please refer to <https://hsdes.intel.com/rest/doc/> for all API endpoints.

These samples are verified to work in both python3.x and python 2.7.

Note: if you are using Python 2.7.x; [please refer to this](#)

Windows

Install Python

<https://www.python.org/downloads/windows/>

Install **requests** module

Open up a command prompt and type in the following

```
py -m pip --proxy https://proxy-dmz.intel.com:912 install requests
```

If you get an error that the proxy only supports http and not https try the following port with http.

```
py -m pip --proxy http://proxy-dmz.intel.com:911 install requests
```

Install **requests-kerberos** module

```
py -m pip --proxy https://proxy-dmz.intel.com:912 install requests-kerberos
```

Set up SSL verification

1) Locate the pem file used by certifi in your env:

```
import certifi
```

```
print(certifi.where())
```

2) download the internal cert (base64 version).

<https://certificates.intel.com/Trust>

Internal Root certs

- Intel SHA384 Root Certificate [\(DER\)](#) [\(Base64\)](#)
- Intel SHA256 Root Certificate [\(DER\)](#) [\(Base64\)](#)
- LEGACY: Intel SHA1 Root Certificate [\(DER\)](#) [\(Base64\)](#)

3) Open the certificates in a text editor and append the entire contents to the pem file located in step 1.

It should look similar to the follow. You must retain -----BEGIN CERTIFICATE-----, -----END CERTIFICATE-----, and all the data in between. You can name the pem file "cacert.pem".

```
-----BEGIN CERTIFICATE-----
MIIGBTCCA+2gAwIBAgIUZaGaZtMW/DbJmSppGwFBCuZ06+EwDQYJKoZIhvcNAQEM
BQAwgYkxHTABBgNVBAMTFEludGVsIFNlQTM4NCBSb290IENBMQswCQYDVQGEwJV
...
<get entire contents from file IntelSHA384RootCA-Base64.crt>
...
9+a0gx4Tq6VzySWFIbZX7r4yLEu5bKrg1Y6Q8CwfXNqbIbRA1uPbNAYZZEI6+JUL
dytE3XyhQ85B
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
6LmNf73o2yTPlhGhj9b/zEKrKSCrkWk5CM08B1ih0h+f0nhF+UtoXkceUDmIVsYV
k7xYdoqXXDbr5TmMX6lcsC8XHj0MF0zFcZTwVr9nJw8ssOK8s3+t+K14qf33SibJ
...
<get entire contents from IntelSHA256RootCA-Base64.crt>
...
I26hieAkPXicvnGc+EVRLRCn5GZrdfrTBZ8KVak4E9lXmq+30hZ326schF0ZNVTY
geY4cbiNt4uaLKVzYlQu327psq5ZEBjjR/aurhbaJWA2YeMY2SB8I2chNHNhJp8=
-----END CERTIFICATE-----
```

Linux

Installation

[Python using Virtual ENV](#)

[Python on standard EC Linux machine](#)

Authorization

On Linux; make sure you run **kinit** to get a kerberos token -> /usr/intel/bin/kinit

You can also run **klist** to see what kerberos tickets you have -> /usr/bin/klist

Sample Calls

How to do Basic Auth in Python

While using HSDDES Basic auth to invoke HSDDES APIs ; do ensure you use /rest/**auth**/article.

```
# GET SERVICE TOKEN - USE INSTRUCTIONS ON HSDES WIKI #
user = "your_name"
pwd = "your_token"
tok = user+':'+pwd
encoded_tok = base64.b64encode(tok.encode()).decode()
url = 'https://hsdes-api.intel.com/rest/auth/article/<articleId>'
headers = {'Content-Type': 'application/json', 'Authorization': 'Basic %s' % encoded_tok}
proxy = {'http': '', 'https': ''}
r = requests.get(URL, headers=headers, proxies=proxy, verify='<path of the .pem>')
```

Kerberos auth for faceless account:

1. Kinit faceless@GAR.CORP.INTEL.COM
2. Klist - here the path for reference will be stored. Pass the path to os.environ similar to the code below.

Note: Please have a script of your choice to create the auth every 12 hours as the key expires after that.

Sample code :

```
import UsrIntel.R1
import requests
from requests_kerberos import HTTPKerberosAuth
import urllib3,os

os.environ['KRB5CCNAME'] = '/tmp/krb5cc_12233253' #should be refreshed every 12 hours
headers = { 'Content-type': 'application/json' }
# Replace the ID here with some article ID
url = 'https://hsdes-api.intel.com/rest/article/2202652508'
proxy = {'http': '', 'https': ''}
response = requests.get(url,auth=HTTPKerberosAuth(),verify="/etc/ssl/certs/" , headers = headers,
proxies=proxy)
print(HTTPKerberosAuth())
print(response.status_code)
print(response)
print(response.json())
```

GET

Get an HSD Article

```
import requests
from requests_kerberos import HTTPKerberosAuth
import urllib3

headers = { 'Content-type': 'application/json' }
# Replace the ID here with some article ID
url = 'https://hsdes-api.intel.com/rest/article/2202652508'
response = requests.get(url, verify='<path of the .pem>', auth=HTTPKerberosAuth(), headers = headers)
print(response.json())
```

Run a query by ID

```
#!/usr/intel/bin/python3.6.3a
import UsrIntel.R1

import requests
from requests_kerberos import HTTPKerberosAuth
import urllib3

headers = { 'Content-type': 'application/json' }
# Replace the ID here with the query ID you want to pull
# Note: the query ID must be a valid ID
url = 'https://hsdes-api.intel.com/rest/query/execution/16010897558'
response = requests.get(url, verify='<path of the .pem>', auth=HTTPKerberosAuth(), headers = headers)
if (response.status_code == 200):
    data_rows = response.json()['data']
    for row in data_rows:
        # Note: you can pull any field thats defined in the query select list
        print(row['id'])
```

Run a query using EQL

change your eql as needed

[Paging Example](#)

[More details on EQL \(ES Query Language\)](#)

```
import requests
from requests_kerberos import HTTPKerberosAuth
import urllib3

headers = { 'Content-type': 'application/json' }

url = 'https://hsdes-api.intel.com/rest/query/execution/eql?start_at=1'

# Note -> make your queries targeted to a tenant or subject for better performance
# in this case; i'm JUST search in tenant = discovery AND subject = issue

payload = """
{
  "eql": "select id,description where discovery.issue.id = 123"
}
"""

response = requests.post(url, verify='<path of the .pem>', auth=HTTPKerberosAuth(), headers = headers, data =
payload)

if (response.status_code == 200):
    data_rows = response.json()['data']
    for row in data_rows:
        print(row['id'])
```

POST

Insert an new HSD article

```
import requests
from requests_kerberos import HTTPKerberosAuth
import urllib3

headers = { 'Content-type': 'application/json' }

# Note we are using the PRE-PRODUCTION URL
url = 'https://hsdes-api-pre.intel.com/rest/article'
payload = """
{
  "subject": "issue",
  "tenant": "discovery",
  "fieldValues": [
    {
      "title": "Testing ES Python API sample"
    },
    {
      "send_mail": "false"
    },
    {
      "owner": "nsshergi"
    }
  ]
}
"""

response = requests.post(url, verify='<path of the .pem>', auth=HTTPKerberosAuth(), headers = headers, data =
payload)
returnValue = response.json()

print ("New ID: " , returnValue["new_id"])
```

Insert a new comment for an existing Article

```

import requests
from requests_kerberos import HTTPKerberosAuth

headers = { 'Content-type': 'application/json' }

# Note we are using the PRE-PRODUCTION URL
url = 'https://hsdes-api-pre.intel.com/rest/article'
payload = """
{
  "subject": "comments",
  "tenant": "discovery",
  "fieldValues": [
    {
      "description": "Testing ES Python API sample comment"
    },
    {
      "parent_id": 1308119657
    },
    {
      "send_mail": "false"
    }
  ]
}
"""

response = requests.post(url, verify='<path of the .pem>', auth=HTTPKerberosAuth(), headers = headers, data =
payload)
returnValue = response.json()

print ("New Comment: " , returnValue["new_id"])

```

[Update an existing HSD article](#)

```

import requests
from requests_kerberos import HTTPKerberosAuth

headers = { 'Content-type': 'application/json' }

# Note we are using the PRE-PRODUCTION URL
url = 'https://hsdes-api-pre.intel.com/rest/article/1307389436'
payload = """
{
  "tenant": "discovery",
  "subject": "issue",
  "fieldValues": [
    {
      "title": "testing from python API"
    },
    {
      "send_mail": "false"
    }
  ]
}
"""

# Note the send_mail false. This will prevent HSD-ES from sending an email for this update.

response = requests.put(url, verify='<path of the .pem>', auth=HTTPKerberosAuth(), headers = headers, data =
payload)
print (response.text)

```

Upload an attachment

Sample code provided by Relihan, David <david.relihan@intel.com>

```

HSDS_PREPROD_SERVER = "https://hsdes-pre.intel.com/rest"
hsdes = HSDSRest(logger = self.logger, tenant = TENANT, server = HSDS_PREPROD_SERVER, username = hsd_idsid,
domain = hsd_domain)
hsd_attachment_id = hsdes_rest.upload_attachment(id = hsd_id, file_name = file_name, file_data = file_data,
file_description = file_description)

```

Here is HSDSRest with all the irrelevant methods removed:

```

import requests

from requests_kerberos import HTTPKerberosAuth, OPTIONAL

import json
import logging
import platform
import subprocess

import requests
import urllib3
import urllib
from requests_kerberos import HTTPKerberosAuth, REQUIRED
from posixpath import join as urljoin
from urllib.parse import urlparse, urlunparse

```

```

urllib3.disable_warnings()

class HSDESRest(object):
    def __init__(self, logger, tenant, server, username, domain):
        self.logger = logger
        self.tenant = tenant
        self.server = server
        self.username = username
        self.domain = domain
        self.principal = f"{username}@{domain}"

    def _auth(self):
        return HTTPKerberosAuth(mutual_authentication=REQUIRED,
                                principal=self.principal,
                                force_preemptive=True)

    def upload_attachment(self, id, file_name, file_data, file_description, send_mail = "false"):
        #url = "https://hsdes-pre.intel.com/rest/binary/upload/1305489307"
        url = self.build_url(self.server, path="binary/upload/" + id)

        payload = {
            "file_name": file_name,
            "title": file_description,
            "send_mail": send_mail
        }

        file = {file_name: file_data}
        r = requests.post(url=url,
                          data=payload,
                          files = file,
                          auth=self._auth(),
                          verify='<path of the .pem>')

        self.logger.info(r)
        if not r.ok:
            raise RuntimeError("Unable to create branch - unexpected HTTP response: %s: %s" % (r.status_code, r.
text))

        self.logger.info (r.text)
        returnJSON = r.json()
        returnVal = None
        if "newID" in returnJSON:
            returnVal = str(returnJSON["newID"])

        self.logger.info ("Attachment HSD ID: " + returnVal)

        return returnVal

    def build_url(self, server, path, args=None):
        """
        If path starts with a leading / then it overrides the path that 'server'
        may contain. Otherwise, the server's path is joined with 'path'.
        """
        if args is None:
            args = {}
        (scheme, netloc, root_path, params, _, fragment) = urlparse(server)
        # posixpath.join (urljoin) does what we want here:
        # If path starts with a leading slash, then root_path is ignored.
        # Otherwise, the two values are joined by a '/'
        path = urljoin(root_path, path)
        query = urllib.parse.urlencode(args)
        return urlunparse([scheme, netloc, path, params, query, fragment])

```

Download an attachment

Code provided by Jaber, Karam <karam.jaber@intel.com>

```
import json
import requests
from requests_kerberos import HTTPKerberosAuth

def download_attachment(file_hsdес_name, file_id):
    """
    :param file_hsdес_name: the name of the file in HSDес
    :param file_id: id of the file in HSDес
    :return:
    """
    headers = { 'Content-type': 'application/json' }
    url = 'https://hsdes-api.intel.com/rest/binary/' + file_id
    response = requests.get(url, verify='<path of the .pem>', auth=HTTPKerberosAuth(), headers = headers)
    with open(file_hsdес_name, "wb") as w:
        w.write(response.content)

def get_attachments_metadata(hsdес_id, hsdес_tenant):
    """
    :param hsdес_tenant: HSDес tenant
    :param hsdес_id: HSDес id
    :return: json list where every element is attachment metadata
    """
    headers = { 'Content-type': 'application/json' }
    url = 'https://hsdes-api.intel.com/rest/article/{}/children'.format(hsdес_id)
    payload = {'tenant': hsdес_tenant, 'child_subject': 'attachment'}
    response = requests.get(url, verify='<path of the .pem>', auth=HTTPKerberosAuth(), headers = headers,
params=payload)
    attachments_list = response.json()["data"]
    return attachments_list

sighting_id = "1234567"
tenant = 'your-hsdes-tenant-name'
attachments_list = get_attachments_metadata(sighting_id, tenant)
download_attachment(attachments_list[0]["title"], attachments_list[0]["id"])
```

Adding embedded images to description

Code provided by kalyanaraman, janani

```
for filepath in glob.iglob('images\\*'):
    with open(filepath, "rb") as image_file:
        encoded_string = base64.b64encode(image_file.read())
        ftext = os.path.splitext(filepath)[1][1:]
        if ftext == 'svg':
            desc= desc + "<img src='data:image/svg+xml;base64," + encoded_string.decode("utf-8") + "'
/>"
        else:
            desc= desc+ "<img src='data:image/png;base64," + encoded_string.decode("utf-8") + "'/>"
```

This will embed all the images in the images/ folder to the end of description text.

desc to be passed to the description in REST API.

Also, import base64 to your py.

Support

If you get stuck on any step; feel free to post in our Yammer channel and we can help you. https://www.yammer.com/intel.com/#!/threads/inGroup?type=in_group&feedId=24633253888